

1 Abstract

This report details the initial phase of a data quality assessment for high-frequency HPLC UV chromatographic data. The primary objective was to characterize signal integrity, specifically addressing a non-zero baseline floor and high-frequency noise that complicates automated peak detection. Analysis of the raw dataset revealed that while the data was complete regarding missing values for the primary variables of interest, the signals exhibited substantial variability (Standard Deviations of 391.61 and 504.03 mAU) relative to their means. Crucially, the baseline was identified as having a negative floor of approximately -7.95 mAU, contradicting standard zero-baseline assumptions. The initial analysis successfully confirmed data completeness but halted at the descriptive statistics stage. A critical methodological gap was identified: the calculation of the 'Average Rolling Standard Deviation' failed, resulting in a missing (NaN) value. This failure prevents the execution of subsequent dynamic threshold optimization and spike removal algorithms, which rely on a valid noise profile. Consequently, the current report concludes that the signal preprocessing pipeline has not yet reached the stage of robust peak detection validation.

2 Introduction

High-performance liquid chromatography (HPLC) relies on precise signal processing to distinguish analyte peaks from baseline noise. In high-frequency UV chromatographic data, traditional assumptions of a zero baseline often fail, particularly in complex biological or process control environments where the baseline may drift to negative values. This study focuses on the evaluation of signal integrity for two primary UV channels: 'UV_1.280_mAU' and 'UV_2.260_mAU'. The dataset comprises over 138,000 time-series points, presenting significant challenges due to a non-zero baseline floor ranging from -7 to -8 mAU and high standard deviations indicative of substantial noise. The motivation for this analysis is to establish a robust, non-zero baseline noise profile. This profile is essential for defining dynamic detection thresholds that minimize false positives without obscuring true analyte peaks. Furthermore, the analysis aims to validate data quality for downstream biopharmaceutical process control, ensuring that automated algorithms can function reliably despite missing process metadata and incomplete concentration data. The core challenge lies in applying appropriate smoothing techniques, such as Savitzky-Golay filtering, to isolate the true noise floor from baseline drift before implementing transient spike removal strategies.

3 Methodology

The data analysis workflow was structured into three sequential phases to characterize the signal and identify processing bottlenecks.

Phase 1: Signal Preprocessing and Baseline Characterization. The dataset ('chromatography_combined.csv') was loaded, focusing on the continuous signal index ('Unnamed: 0') to bypass missing process metadata. Missing values in 'UV_2.260_mAU' were handled via row exclusion, ensuring the analysis relied solely on continuous data. A Savitzky-Golay filter (window=15, poly=3) was applied to both 'UV_1.280_mAU' and 'UV_2.260_mAU' to smooth the signals and reduce high-frequency noise. The filtered signals were saved to new columns. Key metrics calculated included mean and standard deviation for raw versus filtered signals, the identification of the minimum baseline floor, and the average rolling standard deviation.

Phase 2: Dynamic Threshold Optimization and Spike Removal. This phase was designed to apply a median filter (window=3) to the raw signals and calculate a dynamic threshold defined as 3-4 times the rolling standard deviation, anchored to the non-zero baseline floor. Peak indices were intended to be identified where the signal exceeded this threshold. Visualizations were planned to aggregate data by 'run' or 'column' to ensure readability.

Phase 3: Signal-to-Noise Ratio (SNR) and False Positive Estimation. This phase aimed to calculate the average SNR for the filtered baseline and estimate the false positive rate based on UV magnitude consistency. The analysis would assess the impact of the negative baseline on overall sensitivity.

The execution of this methodology revealed a critical failure in Phase 1. While the filtering and baseline characterization steps were initiated, the calculation of the 'Average Rolling Std Dev' resulted in a NaN value. This indicates a failure in applying the rolling window correctly on the filtered data or a calculation error.

As the dynamic threshold optimization depends entirely on this rolling standard deviation, the subsequent steps of spike removal and SNR estimation could not be validly executed.

4 Results and Discussion

The analysis of the provided data confirms that the dataset was complete for the primary variables of interest, with no missing values present in 'UV_1_280_mAU' or 'UV_2_260_mAU' prior to processing, and zero rows excluded post-processing. However, the results highlight significant statistical challenges inherent in the raw signals. As illustrated in Figure 1, the raw signals exhibit high standard deviations (391.61 mAU for 'UV_1_280_mAU' and 504.03 mAU for 'UV_2_260_mAU') relative to their means, suggesting the presence of substantial high-frequency noise that complicates peak detection.

A critical finding is the baseline characterization. Contrary to the assumption of a zero baseline often used in simple noise modeling, the data reveals a negative floor of approximately -7.95 mAU. This non-zero baseline necessitates a more complex noise modeling approach that accounts for the offset. The Savitzky-Golay filtering (window=15, poly=3) was applied to mitigate this, yet the analysis indicates a critical methodological gap. The reported 'Average Rolling Std Dev' is missing (NaN), likely due to a calculation error or a failure to apply the rolling window correctly on the filtered data.

This missing value is a significant impediment to the proposed workflow. Without a successfully calculated rolling standard deviation, the subsequent steps in the plan—specifically the dynamic threshold optimization ($3\text{-}4 \times \text{std}$) and spike removal—cannot be validly executed. The inability to generate a dynamic noise profile means that the automated peak detection algorithms remain unvalidated. The current results demonstrate that while the data is clean and the initial smoothing steps were attempted, the pipeline has stalled at the descriptive statistics stage. Future iterations must address the rolling standard deviation calculation to proceed with false positive estimation and SNR assessment.

5 Conclusion

The initial data analysis for high-frequency HPLC UV chromatographic data has successfully characterized the signal environment but has not yet achieved the goal of robust peak detection validation. The dataset was confirmed to be complete, yet the signals are plagued by high-frequency noise and a significant negative baseline floor of approximately -7.95 mAU. While Savitzky-Golay filtering was applied to address baseline drift, the critical failure to calculate the 'Average Rolling Standard Deviation' (resulting in a NaN value) has halted the workflow. This methodological gap prevents the establishment of a dynamic noise threshold, rendering subsequent spike removal and false positive estimation impossible. To proceed, the data analysis pipeline must be debugged to successfully compute the rolling standard deviation on the filtered data. Once this is resolved, the dynamic threshold optimization can be applied to define a robust noise profile, enabling the validation of automated peak detection algorithms for downstream biopharmaceutical process control.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter

# Load dataset
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Handle missing values in UV_2_260_mAU by row exclusion (in-place)
```

```

missing_count_pre = df['UV_2_260_mAU'].isna().sum()
df.dropna(subset=['UV_2_260_mAU'], inplace=True)
missing_count_post = df['UV_2_260_mAU'].isna().sum()

# Step 2: Apply Savitzky-Golay filtering
window_length = 15
poly_order = 3

# Extract values and filter
filtered_u1 = savgol_filter(df['UV_1_280_mAU'].values, window_length, poly_order)
filtered_u2 = savgol_filter(df['UV_2_260_mAU'].values, window_length, poly_order)

# Update DataFrame with filtered values
df['Filtered_UV_1_280_mAU'] = filtered_u1
df['Filtered_UV_2_260_mAU'] = filtered_u2

# Step 3: Calculate rolling standard deviation for dynamic noise profile
rolling_std_u1 = df['Filtered_UV_1_280_mAU'].rolling(window=window_length).std()
rolling_std_u2 = df['Filtered_UV_2_260_mAU'].rolling(window=window_length).std()
rolling_std_combined = np.mean([rolling_std_u1.values, rolling_std_u2.values])

# Step 4: Calculate statistics
mean_raw_u1 = df['UV_1_280_mAU'].mean()
std_raw_u1 = df['UV_1_280_mAU'].std()
mean_filtered_u1 = df['Filtered_UV_1_280_mAU'].mean()
std_filtered_u1 = df['Filtered_UV_1_280_mAU'].std()

mean_raw_u2 = df['UV_2_260_mAU'].mean()
std_raw_u2 = df['UV_2_260_mAU'].std()
mean_filtered_u2 = df['Filtered_UV_2_260_mAU'].mean()
std_filtered_u2 = df['Filtered_UV_2_260_mAU'].std()

# Identify minimum baseline floor
min_baseline_u1 = np.min(df['Filtered_UV_1_280_mAU'])
min_baseline_u2 = np.min(df['Filtered_UV_2_260_mAU'])
min_baseline_floor = min(min_baseline_u1, min_baseline_u2)

# Average rolling std dev
avg_rolling_std = np.mean(rolling_std_combined)

# Generate markdown table output
output_lines = [
    "## Missing Value Counts",
    f"Pre-processing: {missing_count_pre}",
    f"Post-processing (row exclusion): {missing_count_post}",
    "",
    "## Signal Statistics",
    f"UV_1_280_mAU:",
    f"Raw Mean: {mean_raw_u1:.2f}, Std Dev: {std_raw_u1:.2f}",
    f"Filtered Mean: {mean_filtered_u1:.2f}, Std Dev: {std_filtered_u1:.2f}",
    f"UV_2_260_mAU:",
    f"Raw Mean: {mean_raw_u2:.2f}, Std Dev: {std_raw_u2:.2f}",
    f"Filtered Mean: {mean_filtered_u2:.2f}, Std Dev: {std_filtered_u2:.2f}",
    "",
    "## Baseline and Noise Profile",
    f"Minimum Baseline Floor: {min_baseline_floor:.2f} mAU",
    f"Average Rolling Std Dev: {avg_rolling_std:.2f} mAU"
]

```

```
# Save output to workspace
with open('/workspace/analysis_results_step1.md', 'w') as f:
    f.write('\n'.join(output_lines))
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import logging

# Configure logging
logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s')
logger = logging.getLogger(__name__)

# Load the specified data file
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Print column names and dtypes to verify data structure
print("DataFrame columns:", df.columns.tolist())
print("DataFrame dtypes:")
print(df.dtypes)
print("\nFirst few rows:")
print(df.head())

# Identify actual column names matching the plan's intent
uv1_candidates = [col for col in df.columns if 'UV_1' in col]
uv2_candidates = [col for col in df.columns if 'UV_2' in col]
peak_candidates = [col for col in df.columns if 'Peak' in col]

# Broaden search for peak column to include 'Index', 'Position', or numeric
# columns
if not peak_candidates:
    peak_candidates = [col for col in df.columns if 'Index' in col]
    if not peak_candidates:
        peak_candidates = [col for col in df.columns if 'Position' in col]
    if not peak_candidates:
        # Fallback to first numeric column
        numeric_cols = [col for col in df.columns if pd.api.types.is_numeric_dtype(df[col])]
        if numeric_cols:
            peak_candidates = numeric_cols

# Select the best candidates (preferably those with numeric data)
if not uv1_candidates:
    raise ValueError("No UV1 columns found in dataframe.")
if not uv2_candidates:
    raise ValueError("No UV2 columns found in dataframe.")
if not peak_candidates:
    raise ValueError("No Peak/Index/Position columns found in dataframe.")

# Prefer columns that look like filtered data (contain 'Filtered')
uv1_col = next((c for c in uv1_candidates if 'Filtered' in c), uv1_candidates[0])
uv2_col = next((c for c in uv2_candidates if 'Filtered' in c), uv2_candidates[0])
peak_col = next((c for c in peak_candidates if 'Indices' in c), peak_candidates[0])

# Debug: Display detected column names
print(f"Detected UV1 column: {uv1_col}")
```

```

print(f"Detected_UV2_column:_{uv2_col}")
print(f"Detected_Peak_column:_{peak_col}")

# Select data using the dynamically found column names
data = df[[uv1_col, uv2_col, peak_col]].copy()
data.columns = [uv1_col, uv2_col, peak_col]

# Define peak regions
window_size = 100

# Calculate noise floor (MAD)
noise_UV_1 = np.median(np.abs(data[uv1_col] - np.median(data[uv1_col])))
noise_UV_2 = np.median(np.abs(data[uv2_col] - np.median(data[uv2_col])))

# Calculate SNR directly on raw data using vectorized operations
# Handle zero noise by setting SNR to NaN
if noise_UV_1 == 0:
    snr_UV_1 = np.full_like(data[uv1_col], np.nan)
    logger.warning("Noise_UV1_is_zero;_SNR_set_to_NaN.")
else:
    snr_UV_1 = data[uv1_col] / noise_UV_1

if noise_UV_2 == 0:
    snr_UV_2 = np.full_like(data[uv2_col], np.nan)
    logger.warning("Noise_UV2_is_zero;_SNR_set_to_NaN.")
else:
    snr_UV_2 = data[uv2_col] / noise_UV_2

# Calculate statistics in a single pass
avg_snr_UV_1 = np.nanmean(snr_UV_1)
avg_snr_UV_2 = np.nanmean(snr_UV_2)

# Estimate False Positive Rate
threshold_UV_1 = noise_UV_1
threshold_UV_2 = noise_UV_2

total_points = len(data)
if total_points <= 0:
    fpr_UV_1 = np.nan
    fpr_UV_2 = np.nan
    logger.warning("Total_points_is_zero_or_negative;_FPR_set_to_NaN.")
else:
    # Vectorized peak region masking using numpy binning
    peak_indices = np.array([p for p in data[peak_col].values if isinstance(p, (
        int, np.integer))])
    is_peak_region = np.zeros(len(data), dtype=bool)
    if len(peak_indices) > 0:
        # Create intervals for peak regions
        intervals = [(int(p) - window_size // 2, int(p) + window_size // 2) for p
            in peak_indices]
        # Vectorize the assignment using numpy binning
        for start, end in intervals:
            start = max(0, start)
            end = min(len(data), end)
            is_peak_region[start:end] = True

# Calculate non-peak points count
non_peak_count = np.sum(~is_peak_region)

```

```

if non_peak_count == 0:
    fpr_UV_1 = np.nan
    fpr_UV_2 = np.nan
    logger.warning("No non-peak points found; FPR set to NaN.")
else:
    count_UV_1 = np.sum((data[uv1_col] > threshold_UV_1) & (~is_peak_region))
    count_UV_2 = np.sum((data[uv2_col] > threshold_UV_2) & (~is_peak_region))

    fpr_UV_1 = (count_UV_1 / non_peak_count) * 100
    fpr_UV_2 = (count_UV_2 / non_peak_count) * 100

# Calculate missing data count
missing_count_pre = data[uv1_col].isna().sum() + data[uv2_col].isna().sum()

# Generate report
report_lines = []
if missing_count_pre > 0:
    report_lines = [
        f"Missing Data Count: {missing_count_pre}",
        f"1) Avg SNR: UV1={avg_snr_UV_1:.2f}, UV2={avg_snr_UV_2:.2f}",
        f"2) Est. False Positive Rate: UV1={fpr_UV_1:.2f}%, UV2={fpr_UV_2:.2f}%",
        f"3) Robustness: Baseline median UV1={np.median(data[uv1_col]):.2f}mAU, UV2={np.median(data[uv2_col]):.2f}mAU."
        f"Negative baseline may reduce sensitivity if not corrected."
    ]
else:
    report_lines = [
        f"1) Avg SNR: UV1={avg_snr_UV_1:.2f}, UV2={avg_snr_UV_2:.2f}",
        f"2) Est. False Positive Rate: UV1={fpr_UV_1:.2f}%, UV2={fpr_UV_2:.2f}%",
        f"3) Robustness: Baseline median UV1={np.median(data[uv1_col]):.2f}mAU, UV2={np.median(data[uv2_col]):.2f}mAU."
        f"Negative baseline may reduce sensitivity if not corrected."
    ]

output_text = '\n'.join(report_lines)
if len(output_text) == 0:
    logger.warning("Report text is empty.")
    output_text = "Error: No data to report."

# Save output
with open('/workspace/SNR_FPR_Report.txt', 'w') as f:
    f.write(output_text)

# Print report
print(output_text)

```

Listing 2: Code.3